Aula 4 — EDA Profissional (Versão Avançada) - Para Aula

Dataset: Airbnb NYC

Objetivos Desta Aula

- **Explorar um dataset real de mercado:** Entender a estrutura e o conteúdo de dados reais.
- **Detectar e tratar valores faltantes e outliers:** Aprender técnicas para limpar e preparar os dados.
- **Realizar análise por categoria e por localização:** Investigar padrões e tendências em diferentes segmentos e regiões.
- **Construir gráficos profissionais e informativos:** Utilizar bibliotecas avançadas para visualizações de alta qualidade.
- **Introduzir correlação entre variáveis:** Entender as relações entre diferentes atributos do dataset.
- **Engenharia de Features:** Criar novas variáveis para enriquecer a análise.
- **Análise Geoespacial:** Visualizar dados em um contexto geográfico.
- **Interpretar e comunicar insights:** Extrair conclusões significativas dos dados.

1) Importação das bibliotecas Essenciais

Nesta seção, importamos todas as bibliotecas Python necessárias para a Análise Exploratória de Dados (EDA).

- `pandas` : Para manipulação e análise de dados em formato de DataFrame.
- `matplotlib.pyplot` : Para a criação de gráficos estáticos.
- `seaborn` : Baseado no matplotlib, oferece uma interface de alto nível para criar gráficos estatísticos atraentes e informativos.
- `numpy` : Para operações numéricas e matemáticas, especialmente útil para lidar com arrays e cálculos estatísticos.
- `folium` : Para criar mapas interativos e visualizar dados geoespaciais.

In [1]:
```
# Importe aqui as bibliotecas necessárias: pandas, matplotlib.pyplot, seaborn, numpy
# Configure os estilos de visualização (sns.set_style, plt.rcParams)
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import folium
```

```
from folium.plugins import MarkerCluster
```

2) Carregar o Dataset

Carregamos o dataset do Airbnb NYC a partir de um arquivo CSV. Este é o primeiro passo para qualquer análise de dados, onde os dados brutos são lidos e transformados em um DataFrame do pandas para fácil manipulação.

In [8]:
```
# Carregue o arquivo CSV "AB_NYC_2019.csv" em um DataFrame do pandas
# Exiba as primeiras linhas do DataFrame para uma visão inicial dos dados
df = pd.read_csv("AB_NYC_2019.csv", sep=",")
print(df.head())
```

```
    id                                              name  host_id  \
0  2539                Clean & quiet apt home by the park     2787
1  2595                             Skylit Midtown Castle     2845
2  3647               THE VILLAGE OF HARLEM....NEW YORK !     4632
3  3831                    Cozy Entire Floor of Brownstone     4869
4  5022  Entire Apt: Spacious Studio/Loft by central park     7192

     host_name neighbourhood_group neighbourhood  latitude  longitude  \
0         John            Brooklyn    Kensington  40.64749  -73.97237
1     Jennifer           Manhattan       Midtown  40.75362  -73.98377
2    Elisabeth           Manhattan        Harlem  40.80902  -73.94190
3  LisaRoxanne            Brooklyn  Clinton Hill  40.68514  -73.95976
4        Laura           Manhattan   East Harlem  40.79851  -73.94399

        room_type  price  minimum_nights  number_of_reviews last_review  \
0     Private room    149               1                  9  2018-10-19
1  Entire home/apt    225               1                 45  2019-05-21
2     Private room    150               3                  0         NaN
3  Entire home/apt     89               1                270  2019-07-05
4  Entire home/apt     80              10                  9  2018-11-19

   reviews_per_month  calculated_host_listings_count  availability_365
0               0.21                               6               365
1               0.38                               2               355
2                NaN                               1               365
3               4.64                               1               194
4               0.10                               1                 0
```

3) Visão Geral e Estrutura dos Dados

Esta seção fornece uma visão geral rápida do dataset, incluindo suas dimensões, tipos de dados e um resumo estatístico. Isso nos ajuda a entender a estrutura dos dados e identificar possíveis problemas, como valores ausentes ou tipos de dados incorretos.

In [3]:
```
# Exiba o número de linhas e colunas do DataFrame (df.shape)
# Exiba informações sobre o DataFrame, incluindo tipos de dados e valores não-nulos
# Exiba estatísticas descritivas para as colunas numéricas (df.describe())
# Exiba estatísticas descritivas para todas as colunas (incluindo categóricas) (df.
```

```python
lin, col = df.shape
print("\n" + "-" * 60)
print(f"Numero Linhas: {lin}")
print(f"Numero Colunas: {col}")
print("-" * 60)

print("\n" + "-" * 60)
print("\nInformações do DataFrame: ")
print("-" * 60)
df.info()

print("\n" + "-" * 60)
print("\nEstatisticas descritivas (colunas numericas): ")
print(df.describe())
print("-" * 60)

print("\n" + "-" * 60)
print("\nEstatisticas descritivas (todas colunas): ")
print(df.describe(include="all"))
print("-" * 60)
```

```
------------------------------------------------------------
Numero Linhas: 48895
Numero Colunas: 16
------------------------------------------------------------

------------------------------------------------------------

Informações do DataFrame:
------------------------------------------------------------
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              48895 non-null  int64
 1   name                            48879 non-null  object
 2   host_id                         48895 non-null  int64
 3   host_name                       48874 non-null  object
 4   neighbourhood_group             48895 non-null  object
 5   neighbourhood                   48895 non-null  object
 6   latitude                        48895 non-null  float64
 7   longitude                       48895 non-null  float64
 8   room_type                       48895 non-null  object
 9   price                           48895 non-null  int64
 10  minimum_nights                  48895 non-null  int64
 11  number_of_reviews               48895 non-null  int64
 12  last_review                     38843 non-null  object
 13  reviews_per_month               38843 non-null  float64
 14  calculated_host_listings_count  48895 non-null  int64
 15  availability_365                48895 non-null  int64
dtypes: float64(3), int64(7), object(6)
memory usage: 6.0+ MB

------------------------------------------------------------

Estatisticas descritivas (colunas numericas):
                 id       host_id      latitude     longitude         price  \
count  4.889500e+04  4.889500e+04  48895.000000  48895.000000  48895.000000
mean   1.901714e+07  6.762001e+07     40.728949    -73.952170    152.720687
std    1.098311e+07  7.861097e+07      0.054530      0.046157    240.154170
min    2.539000e+03  2.438000e+03     40.499790    -74.244420      0.000000
25%    9.471945e+06  7.822033e+06     40.690100    -73.983070     69.000000
50%    1.967728e+07  3.079382e+07     40.723070    -73.955680    106.000000
75%    2.915218e+07  1.074344e+08     40.763115    -73.936275    175.000000
max    3.648724e+07  2.743213e+08     40.913060    -73.712990  10000.000000

       minimum_nights  number_of_reviews  reviews_per_month  \
count    48895.000000       48895.000000       38843.000000
mean         7.029962          23.274466           1.373221
std         20.510550          44.550582           1.680442
min          1.000000           0.000000           0.010000
25%          1.000000           1.000000           0.190000
50%          3.000000           5.000000           0.720000
75%          5.000000          24.000000           2.020000
max       1250.000000         629.000000          58.500000
```

```
       calculated_host_listings_count  availability_365
count                    48895.000000      48895.000000
mean                         7.143982        112.781327
std                         32.952519        131.622289
min                          1.000000          0.000000
25%                          1.000000          0.000000
50%                          1.000000         45.000000
75%                          2.000000        227.000000
max                        327.000000        365.000000
------------------------------------------------------------

------------------------------------------------------------

Estatisticas descritivas (todas colunas):
                  id            name       host_id host_name  \
count   4.889500e+04           48879  4.889500e+04     48874
unique           NaN           47905           NaN     11452
top              NaN  Hillside Hotel           NaN   Michael
freq             NaN              18           NaN       417
mean    1.901714e+07             NaN  6.762001e+07       NaN
std     1.098311e+07             NaN  7.861097e+07       NaN
min     2.539000e+03             NaN  2.438000e+03       NaN
25%     9.471945e+06             NaN  7.822033e+06       NaN
50%     1.967728e+07             NaN  3.079382e+07       NaN
75%     2.915218e+07             NaN  1.074344e+08       NaN
max     3.648724e+07             NaN  2.743213e+08       NaN

       neighbourhood_group neighbourhood      latitude     longitude  \
count                48895         48895  48895.000000  48895.000000
unique                   5           221           NaN           NaN
top              Manhattan  Williamsburg           NaN           NaN
freq                 21661          3920           NaN           NaN
mean                   NaN           NaN     40.728949    -73.952170
std                    NaN           NaN      0.054530      0.046157
min                    NaN           NaN     40.499790    -74.244420
25%                    NaN           NaN     40.690100    -73.983070
50%                    NaN           NaN     40.723070    -73.955680
75%                    NaN           NaN     40.763115    -73.936275
max                    NaN           NaN     40.913060    -73.712990

              room_type         price  minimum_nights  number_of_reviews  \
count             48895  48895.000000    48895.000000       48895.000000
unique                3           NaN             NaN                NaN
top     Entire home/apt           NaN             NaN                NaN
freq              25409           NaN             NaN                NaN
mean                NaN    152.720687        7.029962          23.274466
std                 NaN    240.154170       20.510550          44.550582
min                 NaN      0.000000        1.000000           0.000000
25%                 NaN     69.000000        1.000000           1.000000
50%                 NaN    106.000000        3.000000           5.000000
75%                 NaN    175.000000        5.000000          24.000000
max                 NaN  10000.000000     1250.000000         629.000000

       last_review  reviews_per_month  calculated_host_listings_count  \
count        38843       38843.000000                    48895.000000
unique        1764                NaN                             NaN
```

```
top     2019-06-23                NaN                            NaN
freq          1413                NaN                            NaN
mean           NaN           1.373221                       7.143982
std            NaN           1.680442                      32.952519
min            NaN           0.010000                       1.000000
25%            NaN           0.190000                       1.000000
50%            NaN           0.720000                       1.000000
75%            NaN           2.020000                       2.000000
max            NaN          58.500000                     327.000000

        availability_365
count       48895.000000
unique               NaN
top                  NaN
freq                 NaN
mean          112.781327
std           131.622289
min             0.000000
25%             0.000000
50%            45.000000
75%           227.000000
max           365.000000
------------------------------------------------------------
```

4) Tratamento de Valores Faltantes

Valores faltantes são comuns em datasets reais e podem afetar a qualidade da análise. Nesta seção, identificamos e tratamos esses valores, decidindo entre remover, preencher com um valor específico (média, mediana, moda) ou usar métodos mais avançados.

In [9]:

```
# Calcule e exiba a quantidade de valores faltantes por coluna
# Preencha os valores faltantes nas colunas `name` e `host_name` com "unknown"
# Preencha os valores faltantes na coluna `reviews_per_month` com 0
# Converta a coluna `last_review` para datetime e preencha os valores faltantes com
# Verifique novamente os valores faltantes após o tratamento

print("\n" + "-" * 60)
print("Valores faltantes por coluna antes do tratamento: ")
print(df.isna().sum())
print("-" * 60)

df["name"] = df["name"].fillna("unknown")
df["host_name"] = df["host_name"].fillna("unknown")

df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

df["last_review"] = pd.to_datetime(df["last_review"])
df["last_review"] = df["last_review"].fillna("2010-01-01")

print("\n" + "-" * 60)
print("Colunas faltantes apos tratamento: ")
print(df.isna().sum())
print("-" * 60)
```

```
------------------------------------------------------------
Valores faltantes por coluna antes do tratamento:
id                                   0
name                                16
host_id                              0
host_name                           21
neighbourhood_group                  0
neighbourhood                        0
latitude                             0
longitude                            0
room_type                            0
price                                0
minimum_nights                       0
number_of_reviews                    0
last_review                      10052
reviews_per_month                10052
calculated_host_listings_count       0
availability_365                     0
dtype: int64
------------------------------------------------------------

------------------------------------------------------------
Colunas faltantes apos tratamento:
id                               0
name                             0
host_id                          0
host_name                        0
neighbourhood_group              0
neighbourhood                    0
latitude                         0
longitude                        0
room_type                        0
price                            0
minimum_nights                   0
number_of_reviews                0
last_review                      0
reviews_per_month                0
calculated_host_listings_count   0
availability_365                 0
dtype: int64
------------------------------------------------------------
```

5) Detecção e Tratamento de Outliers

Outliers são pontos de dados que se desviam significativamente de outras observações. Eles podem distorcer análises estatísticas e modelos de machine learning. Nesta seção, identificamos e tratamos outliers em variáveis numéricas chave, como `price` e `minimum_nights`. Usaremos o método do Intervalo Interquartil (IQR) para uma abordagem robusta.

In [11]:
```
# Defina uma função `remove_outliers_iqr` para detectar e remover outliers usando o
# Aplique a função para tratar outliers nas colunas `price`, `minimum_nights` e `nu
# Exiba as estatísticas descritivas antes e depois do tratamento para cada coluna
```

```python
# Atualize o DataFrame principal com os dados limpos
def remove_outliers_iqr(df, col):

    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)

    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    return df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

print("\n" + "-" * 60)
print("Estatisticas antes do tratamento: ")
print(df[["price", "minimum_nights", "number_of_reviews"]].describe())
print("-" * 60)

df = remove_outliers_iqr(df, "price")
df = remove_outliers_iqr(df, "minimum_nights")
df = remove_outliers_iqr(df, "number_of_reviews")

print("\n" + "-" * 60)
print("Estatisticas depois do tratamento: ")
print(df[["price", "minimum_nights", "number_of_reviews"]].describe())
print("-" * 60)
```

```
------------------------------------------------------------
Estatisticas antes do tratamento:
              price  minimum_nights  number_of_reviews
count  48895.000000    48895.000000       48895.000000
mean     152.720687        7.029962          23.274466
std      240.154170       20.510550          44.550582
min        0.000000        1.000000           0.000000
25%       69.000000        1.000000           1.000000
50%      106.000000        3.000000           5.000000
75%      175.000000        5.000000          24.000000
max    10000.000000     1250.000000         629.000000
------------------------------------------------------------

------------------------------------------------------------
Estatisticas depois do tratamento:
              price  minimum_nights  number_of_reviews
count  35202.000000    35202.000000       35202.000000
mean     119.289643        2.765042          12.362536
std       67.831566        1.917473          16.631476
min        0.000000        1.000000           0.000000
25%       65.000000        1.000000           1.000000
50%      100.000000        2.000000           5.000000
75%      155.000000        3.000000          17.000000
max      334.000000       11.000000          71.000000
------------------------------------------------------------
```

6) Análise Exploratória de Dados (EDA) Aprofundada

Com os dados limpos, podemos agora aprofundar nossa análise, explorando as distribuições das variáveis, as relações entre elas e identificando padrões que podem gerar insights valiosos.

6.1) Distribuição de Preços

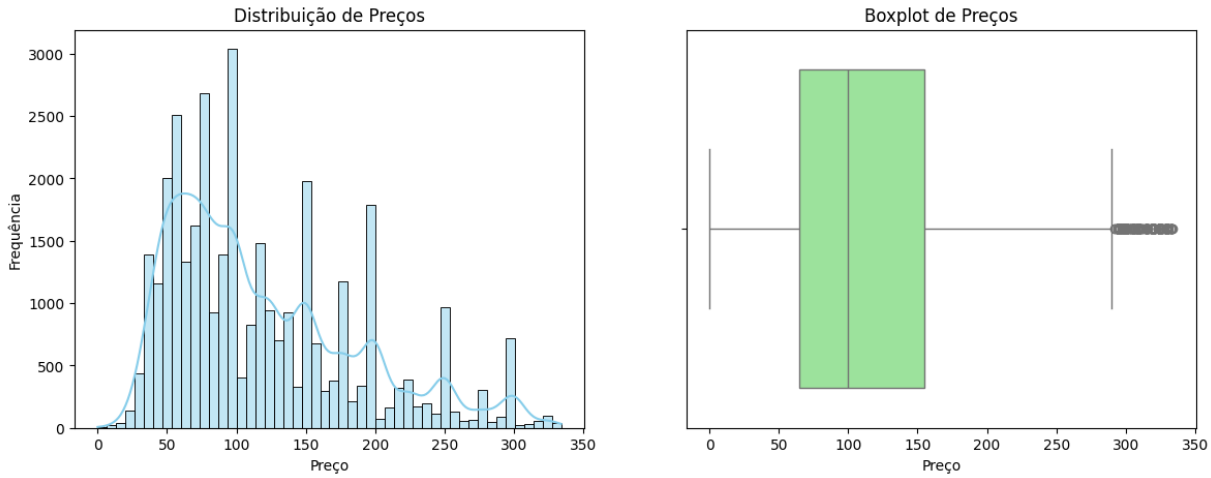
Vamos visualizar a distribuição dos preços dos aluguéis para entender a faixa de valores e identificar a concentração de preços.

In [12]:
```
# Crie um histograma da coluna `price` usando `sns.histplot`
# Crie um boxplot da coluna `price` usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
sns.histplot(df['price'], bins=50, kde=True, color='skyblue')
plt.title('Distribuição de Preços')
plt.xlabel('Preço')
plt.ylabel('Frequência')

plt.subplot(1, 2, 2)
sns.boxplot(x=df['price'], color='lightgreen')
plt.title('Boxplot de Preços')
plt.xlabel('Preço')
```

Out[12]: `Text(0.5, 0, 'Preço')`



6.2) Análise por Tipo de Quarto (room_type)

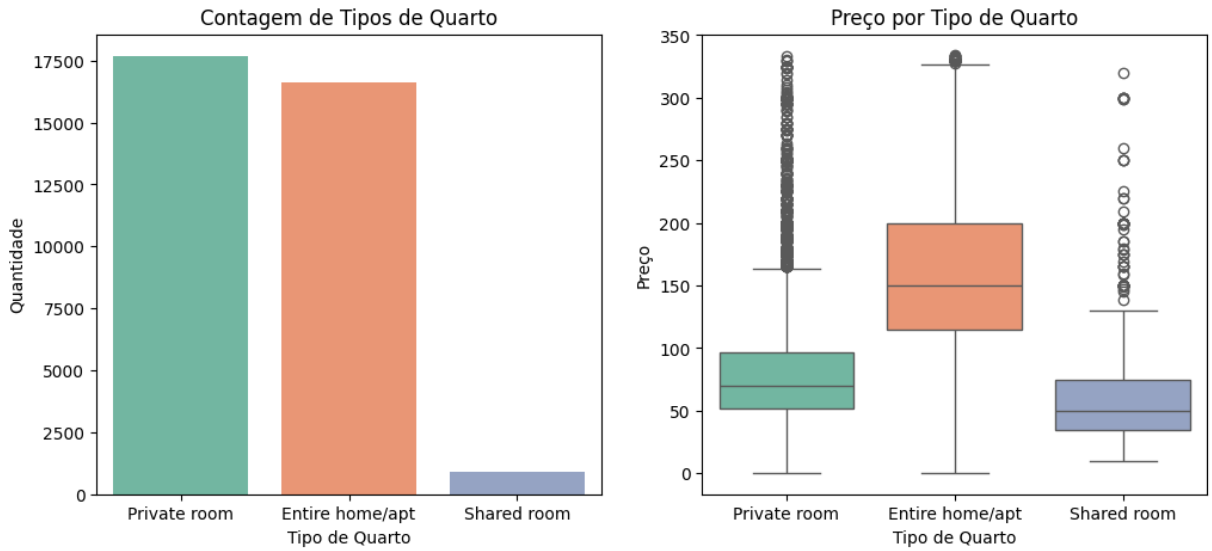
Entender a distribuição dos tipos de quartos e como eles se relacionam com o preço é crucial para proprietários e hóspedes.

In [18]:
```
# Crie um gráfico de contagem dos tipos de quarto usando `sns.countplot`
# Crie um boxplot do preço por tipo de quarto usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
sns.countplot(data=df, x='room_type', hue='room_type', palette='Set2', legend=False
plt.title('Contagem de Tipos de Quarto')
plt.xlabel('Tipo de Quarto')
plt.ylabel('Quantidade')

plt.subplot(1, 2, 2)
sns.boxplot(data=df, x='room_type', y='price', hue='room_type', palette='Set2', leg
plt.title('Preço por Tipo de Quarto')
plt.xlabel('Tipo de Quarto')
plt.ylabel('Preço')
```

Out[18]: Text(0, 0.5, 'Preço')



6.3) Análise por Grupo de Bairro (neighbourhood_group)

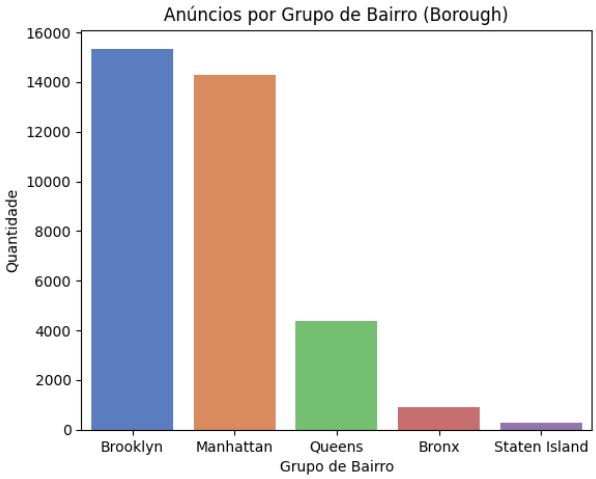
A localização é um dos fatores mais importantes no mercado imobiliário. Vamos analisar como os preços e a disponibilidade variam entre os diferentes grupos de bairros de NYC.

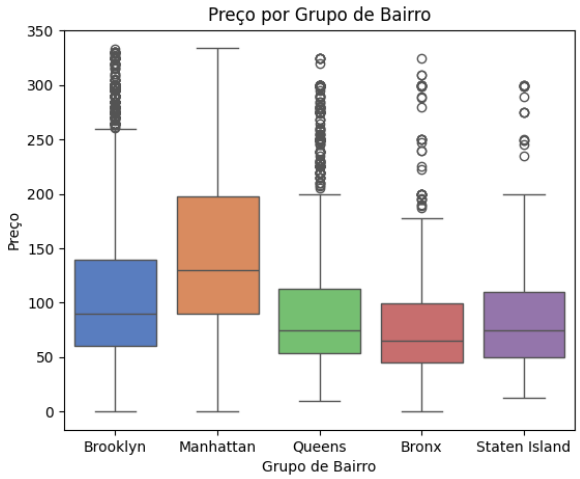
In [20]:
```
# Crie um gráfico de contagem de anúncios por grupo de bairro usando `sns.countplot
# Crie um boxplot do preço por grupo de bairro usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
sns.countplot(data=df, x='neighbourhood_group', hue='neighbourhood_group', palette=
plt.title('Anúncios por Grupo de Bairro (Borough)')
plt.xlabel('Grupo de Bairro')
plt.ylabel('Quantidade')

plt.subplot(1, 2, 2)
sns.boxplot(data=df, x='neighbourhood_group', y='price', hue='neighbourhood_group',
plt.title('Preço por Grupo de Bairro')
plt.xlabel('Grupo de Bairro')
plt.ylabel('Preço')
```

Out[20]: Text(0, 0.5, 'Preço')



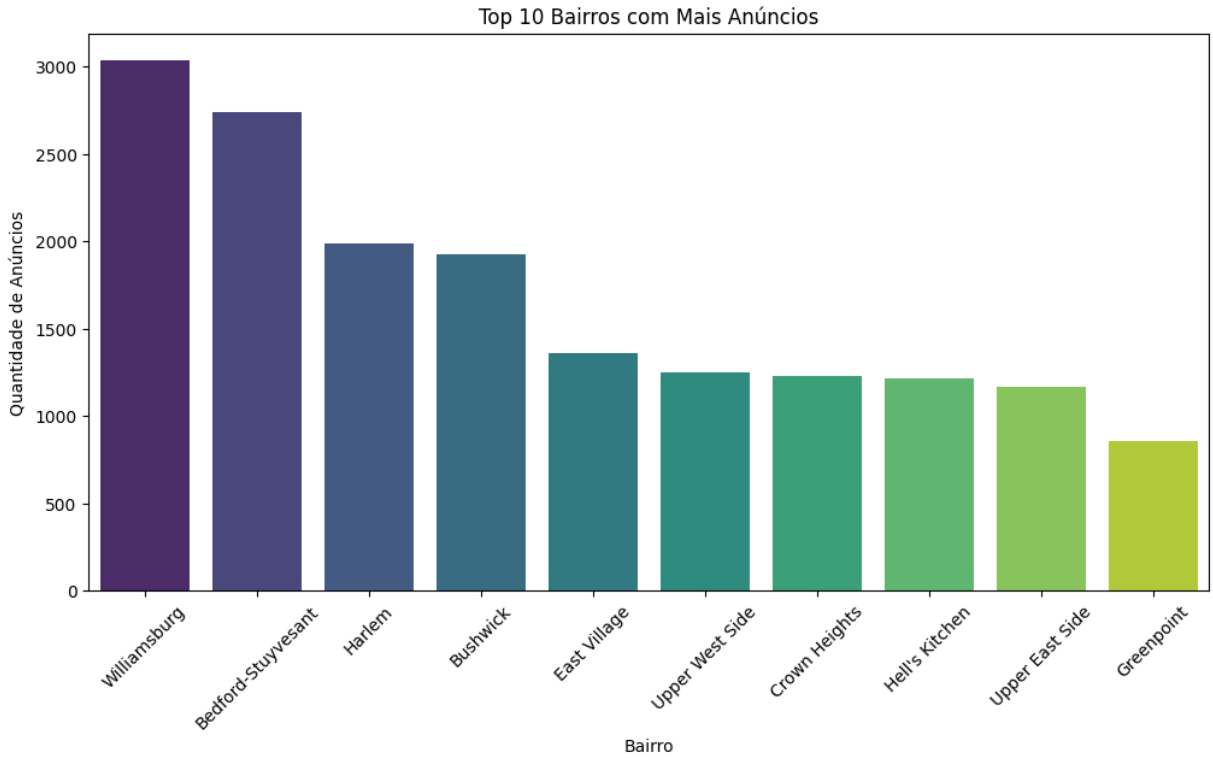


6.4) Top 10 Bairros com Mais Anúncios

Identificar os bairros com maior concentração de anúncios pode indicar áreas de alta demanda ou popularidade.

In [27]:
```
# Calcule os 10 bairros com mais anúncios
# Crie um gráfico de barras para visualizar esses bairros usando `sns.barplot`
# Adicione títulos e rótulos aos gráficos, e rotacione os rótulos do eixo X se nece
top_10_bairros = df['neighbourhood'].value_counts().head(10)

plt.figure(figsize=(12, 6))
sns.barplot(x=top_10_bairros.index, y=top_10_bairros.values, hue=top_10_bairros.ind
plt.title('Top 10 Bairros com Mais Anúncios')
plt.xlabel('Bairro')
plt.ylabel('Quantidade de Anúncios')
plt.xticks(rotation=45);
```



6.5) Análise Geoespacial dos Anúncios

Visualizar os anúncios em um mapa nos permite identificar clusters geográficos e entender a distribuição espacial dos imóveis.

In [31]:
```
# Crie um mapa base de Nova York usando `folium.Map`
# Adicione um `MarkerCluster` ao mapa
# Amostre uma parte do DataFrame (e.g., 5000 pontos) para melhor performance na vis
# Adicione marcadores ao mapa para cada ponto amostrado, incluindo informações no p
# Salve o mapa em um arquivo HTML (e.g., "airbnb_nyc_map.html")
# (Opcional) Exiba o mapa diretamente no notebook se o ambiente suportar
df_amostra = df.sample(n=5000, random_state=42)

nyc_map = folium.Map(location=[40.7128, -74.0060], zoom_start=11)

marker_cluster = MarkerCluster().add_to(nyc_map)

for idx, row in df_amostra.iterrows():
```

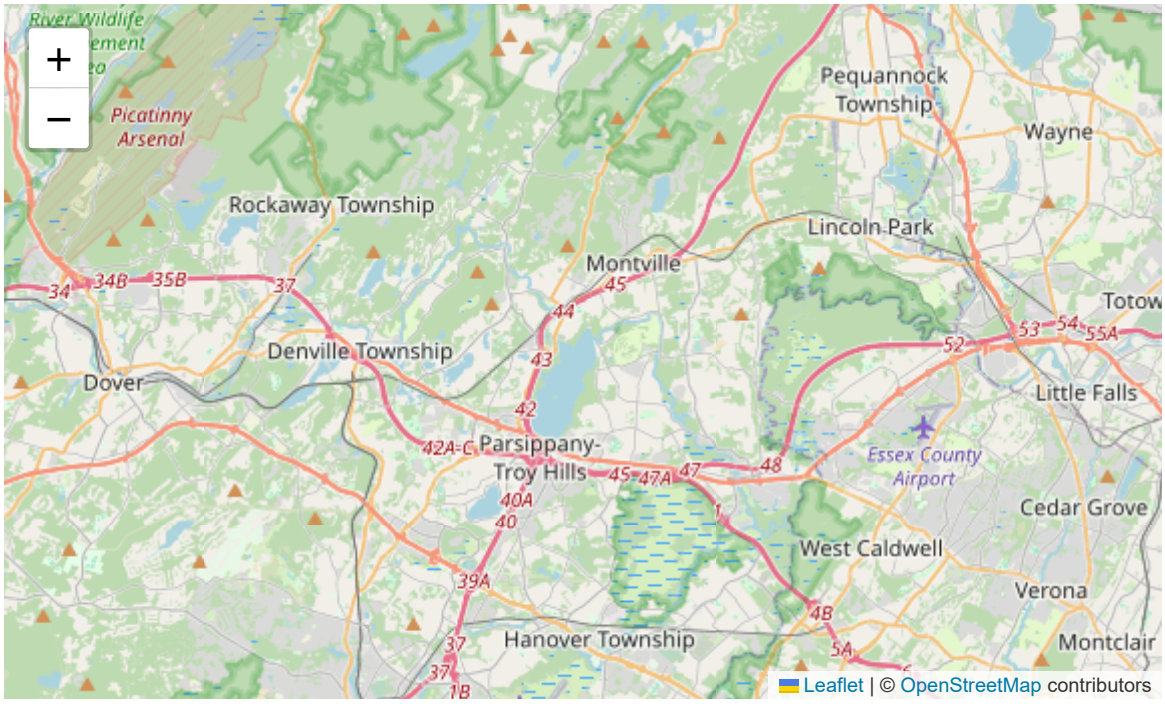
```
    folium.Marker(
        location=[row['latitude'], row['longitude']],
        popup=f"Bairro: {row['neighbourhood']} <br> Preço: ${row['price']} <br> Tip
    ).add_to(marker_cluster)

print("\n" + "-" * 60)
nyc_map.save("airbnb_nyc_map.html")
print("Mapa salvo como 'airbnb_nyc_map.html'")
print("-" * 60)

nyc_map
```

```
------------------------------------------------------------
Mapa salvo como 'airbnb_nyc_map.html'
------------------------------------------------------------
```

Out[31]:



7) Engenharia de Features

A engenharia de features envolve a criação de novas variáveis a partir das existentes para melhorar a capacidade preditiva dos modelos ou para obter novos insights. Aqui, criaremos algumas features simples.

7.1) `host_experience_days`

Calcularemos a experiência do anfitrião em dias, baseando-nos na data da última avaliação (`last_review`) e uma data de referência (ou a data da primeira avaliação, se disponível). Isso pode indicar a longevidade e experiência do anfitrião na plataforma.

In [32]:

```
# Calcule a data mais recente de `last_review`
# Crie a nova feature `host_experience_days` calculando a diferença em dias entre a
# Exiba as estatísticas descritivas da nova feature
```

```
data_mais_recente = df["last_review"].max()
df["host_experience_days"] = (data_mais_recente - df["last_review"]).dt.days

print("\n" + "-" * 60)
print(df["host_experience_days"].describe())
print("-" * 60)
```

```
------------------------------------------------------------
count    35202.000000
mean       912.940032
std       1301.936154
min          0.000000
25%         22.000000
50%        190.000000
75%       1150.000000
max       3475.000000
Name: host_experience_days, dtype: float64
------------------------------------------------------------
```

7.2) `price_per_night_per_review`

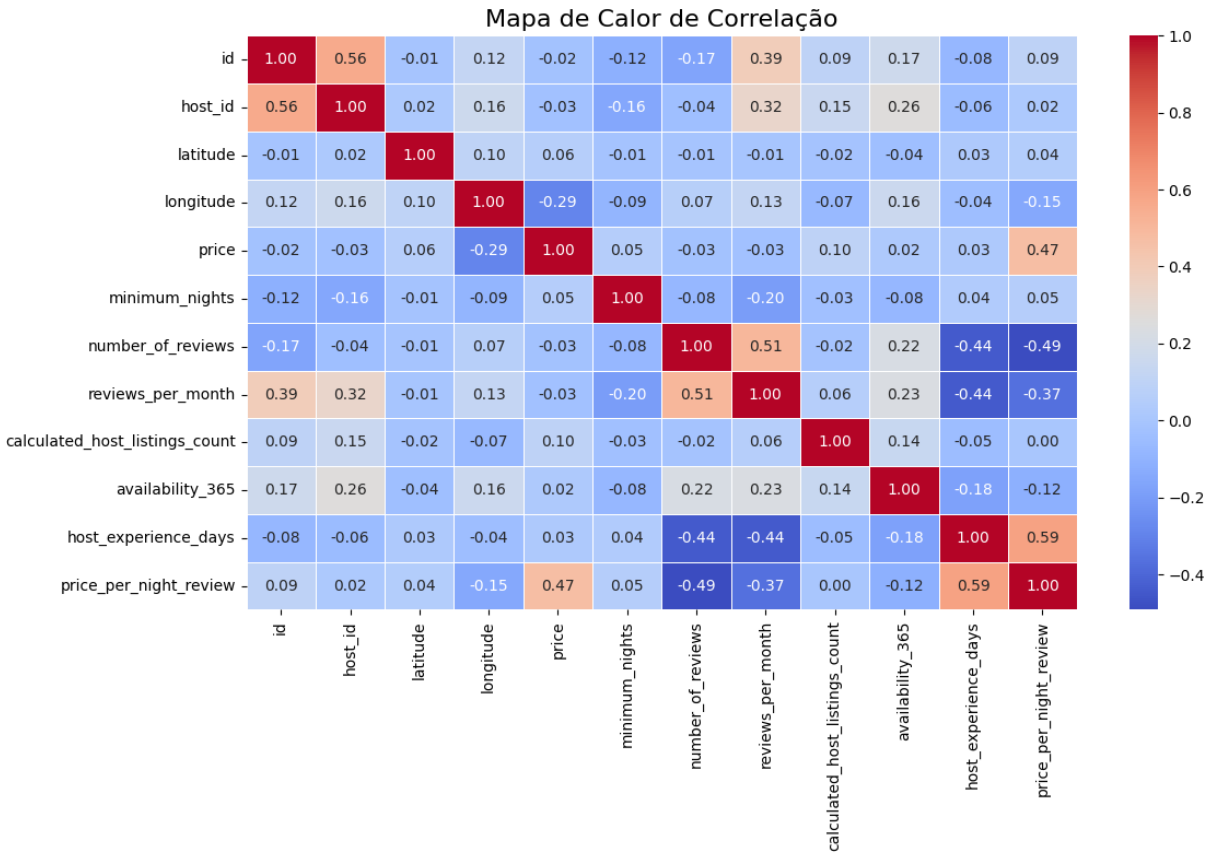
Esta feature pode indicar o custo-benefício percebido, relacionando o preço por noite com o número de avaliações. Um valor baixo pode indicar um bom negócio, enquanto um valor alto pode sugerir um preço elevado para poucos reviews.

In [33]:
```
# Crie a nova feature `price_per_night_per_review`, dividindo `price` por `number_o
# Lembre-se de tratar a divisão por zero (e.g., se `number_of_reviews` for 0, use o
# Exiba as estatísticas descritivas da nova feature
df["price_per_night_review"] = df["price"] / df["number_of_reviews"].replace(0, 1)

print("\n" + "-" * 60)
print(df["price_per_night_review"].describe())
print("-" * 60)
```

```
------------------------------------------------------------
count    35202.000000
mean        49.967902
std         63.791240
min          0.000000
25%          5.801613
50%         21.966667
75%         70.000000
max        333.000000
Name: price_per_night_review, dtype: float64
------------------------------------------------------------
```

8) Análise de Correlação

A correlação mede a força e a direção da relação linear entre duas variáveis numéricas. Um mapa de calor (heatmap) da matriz de correlação é uma excelente ferramenta para visualizar essas relações.

In [35]:
```
# Selecione apenas as colunas numéricas do DataFrame
# Calcule a matriz de correlação
# Crie um heatmap da matriz de correlação usando `sns.heatmap`
# Adicione anotações e um mapa de cores adequado
# Adicione um título ao gráfico
df_numerico = df.select_dtypes(include=[np.number])

matriz_correlacao = df_numerico.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(matriz_correlacao, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0
plt.title('Mapa de Calor de Correlação', fontsize=16)
plt.tight_layout()
plt.show()
```